Nonlocal vs Global Scopes

`global` rebinds a module-level name; `nonlocal` rebinds a variable in the nearest enclosing function.

global — module-level state

```
counter = 0
def hit():
    # rebind the module-level name
    global counter
    counter += 1

hit(); hit()
print(counter)
```

OUTPUT
```
2
```

nonlocal — enclosing-function state

```
def make_counter():
    count = 0
    def step():
        # rebind the enclosing variable
        nonlocal count
        count += 1
        return count
    return step

c = make_counter()
print(c(), c(), c())
# a separate closure, separate state
d = make_counter()
print(d())
```

OUTPUT
```
1 2 3
1
```

Side by side

	`global x`	`nonlocal x`
Refers to	module-level name	nearest enclosing **function** variable
Must exist first?	No – created if missing	Yes – else `SyntaxError`
Typical use	app-wide config / counters	closures, accumulators, decorator state
Reaches module level?	Yes	Never

```
try:
    exec("def f():\n    nonlocal zzz\n    zzz = 1")
except SyntaxError as e:
    print(e.msg)
```

OUTPUT
```
no binding for nonlocal 'zzz' found
```

No declaration = you shadow it

```
level = "global"
def f():
    # new local; the global is untouched
    level = "local"
    return level

print(f(), level)
```

OUTPUT
```
local global
```

Read-only globals are fine

```
# UPPER_CASE = constant by convention
MAX_TRIES = 3
def label(n):
    return f"try {n}/{MAX_TRIES}"

print(label(1))
```

OUTPUT
```
try 1/3
```

You only need them to rebind

```
items = []
def add(x):
    # mutation — no `global` needed
    items.append(x)
add(1); add(2)
print(items)
```

OUTPUT
```
[1, 2]
```

- `x += 1` is an **assignment**, so it needs the declaration; `items.append()` does not.
- Declare `global` / `nonlocal` **before** the first use of the name in that function.

Which one do I need?

- Changing a module-level variable inside a function → `global`.
- Changing an outer function's variable from an inner function → `nonlocal`.
- Only reading it → neither.

PITFALL `global` creates hidden coupling and hard-to-test code. Prefer parameters + return values, or a class / closure that owns the state.

TIP A closure with `nonlocal` is a lightweight alternative to a class with a single attribute.